



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 27-b: Solución Mario Maker

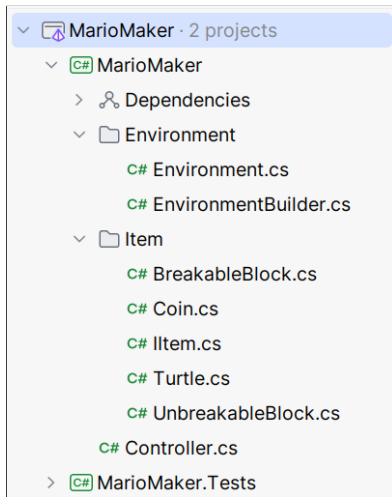
Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC21123 Diseño Detallado de Software

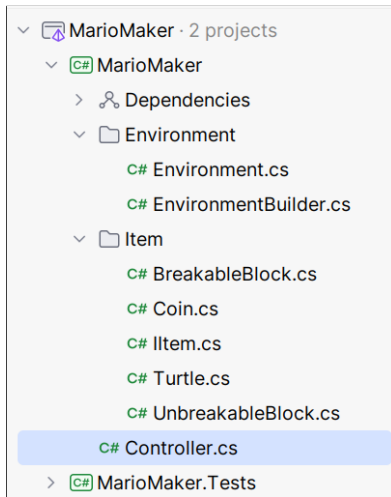
13 de Noviembre, 2025

Solución Mario Maker

Solución Mario Maker



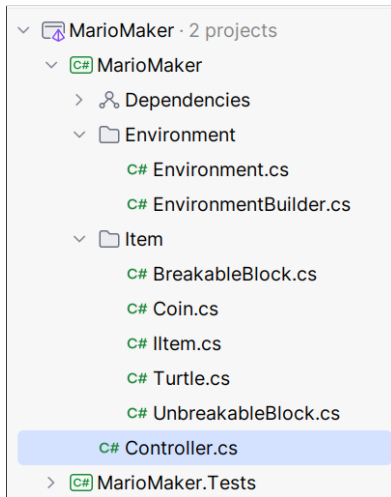
Solución Mario Maker



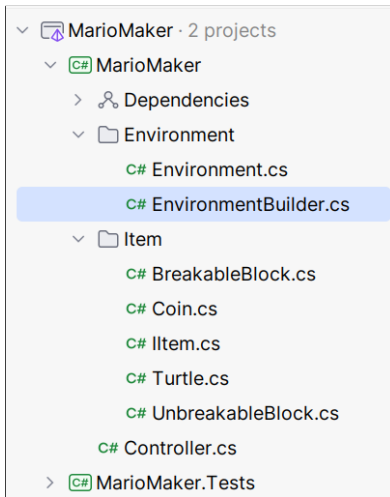
Solución Mario Maker

```
1 public static class Controller {  
2  
3     public static Environment BuildEnvironment(EnvironmentBuilder builder, string[]  
4         lines) {  
5         throw new NotImplementedException();  
6     }  
7  
8     public static EnvironmentBuilder BuildCloudEnvironmentBuilder() {  
9         throw new NotImplementedException();  
10    }  
11  
12    public static EnvironmentBuilder BuildLavaEnvironmentBuilder() {  
13        throw new NotImplementedException();  
14    }  
15  
16    public static string[] Export(Environment environment) {  
17        throw new NotImplementedException();  
18    }  
19 }
```

Solución Mario Maker



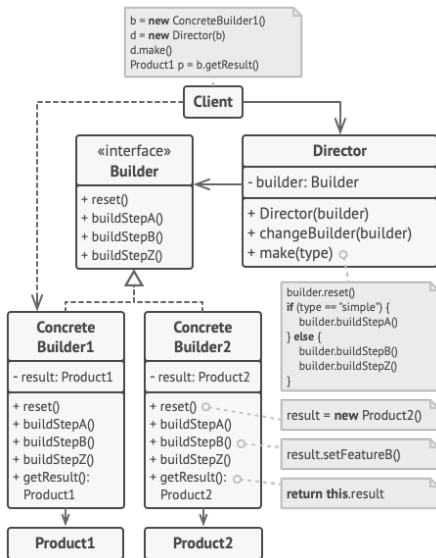
Solución Mario Maker



Solución Mario Maker

```
1 public class EnvironmentBuilder {
2     private Environment _environment;
3
4     public void Reset()
5         => _environment = new Environment();
6
7     public void SetDimensions(int height, int width)
8         => _environment.SetDimensions(height, width);
9
10    public void SetUnbreakableBlock(int row, int column)
11        => _environment.SetItem(row, column, new UnbreakableBlock());
12    public void SetBreakableBlock(int row, int column)
13        => _environment.SetItem(row, column, new BreakableBlock());
14    public void SetTurtle(int row, int column)
15        => _environment.SetItem(row, column, new Turtle());
16    public void SetCoin(int row, int column)
17        => _environment.SetItem(row, column, new Coin());
18
19    public Environment GetResult()
20        => _environment;
21 }
```


Solución Mario Maker



Parte a) Actualmente hay un programa para generar *templates* de niveles, los cuales son archivos de texto que representan una grilla. El número de líneas dentro del archivo representa la altura de la grilla y el largo de las líneas el ancho (puedes asumir que todas las líneas del archivo tienen el mismo largo).

Cada caracter (en caso de haber alguno) representa una entidad del ambiente:

- U: UnbreakableBlock.
- B: BreakableBlock.
- C: Coin.
- T: Turtle.

Implementa el método `BuildEnvironment(EnvironmentBuilder builder, string[] lines)` dentro de `Controller.cs`. Este método debe retornar un objeto del tipo `Environment` que sea construido a partir de un *template*, el cual está dado por el argumento `lines`.

Solución Mario Maker

```
1 public static class Controller {  
2  
3     public static Environment BuildEnvironment(EnvironmentBuilder builder, string[]  
4         lines) {  
5         throw new NotImplementedException();  
6     }  
7  
8     public static EnvironmentBuilder BuildCloudEnvironmentBuilder() {  
9         throw new NotImplementedException();  
10    }  
11  
12    public static EnvironmentBuilder BuildLavaEnvironmentBuilder() {  
13        throw new NotImplementedException();  
14    }  
15  
16    public static string[] Export(Environment environment) {  
17        throw new NotImplementedException();  
18    }  
19 }
```

Solución Mario Maker

```
1 public static Environment BuildEnvironment(EnvironmentBuilder builder, string[]
   lines) {
2     builder.Reset();
3
4     int height = lines.Length;
5     int width = lines[0].Length;
6     builder.SetDimensions(height, width);
7     for (int i = 0; i < height; i++)
8         for (int j = 0; j < width; j++)
9         {
10             if(lines[i][j] == 'U')
11                 builder.SetUnbreakableBlock(i, j);
12             if(lines[i][j] == 'T')
13                 builder.SetTurtle(i, j);
14             if(lines[i][j] == 'C')
15                 builder.SetCoin(i, j);
16             if(lines[i][j] == 'B')
17                 builder.SetBreakableBlock(i, j);
18         }
19
20     return builder.GetResult();
21 }
```

Parte b) Extiende el patrón Builder para generar niveles ambientados en el mundo de lava y en el mundo de nubes. Para ello implementa:

- `BuildLavaEnvironmentBuilder()`
- `BuildCloudEnvironmentBuilder()`

Por ahora, la única diferencia entre estos mundos es el tipo de tortugas que se encuentran en ellos. En el mundo de lava solo existen tortugas de lava y en el mundo de nube solo existen tortugas con alas.

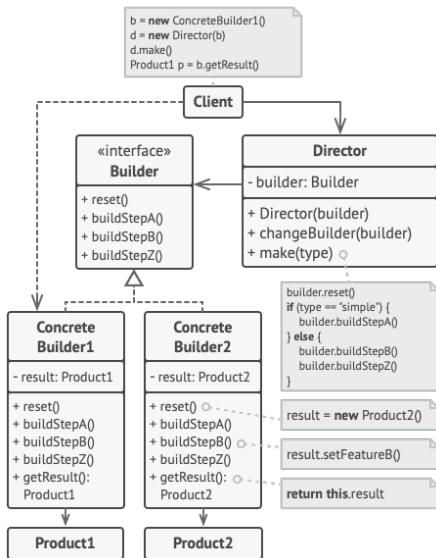
En específico, el método `Export()` de las tortugas del mundo de lava debe retornar “LavaTurtle” y las del mundo de nube debe retornar “WingedTurtle”.

Solución Mario Maker

Nos piden retornar builders para estos nuevos tipos de ambientes:

```
1 public static class Controller {
2
3     public static Environment BuildEnvironment(EnvironmentBuilder builder, string[]
        lines) {
4         /* ... idem anterior ... */
5     }
6
7     public static EnvironmentBuilder BuildCloudEnvironmentBuilder(){
8         throw new NotImplementedException();
9     }
10
11    public static EnvironmentBuilder BuildLavaEnvironmentBuilder(){
12        throw new NotImplementedException();
13    }
14
15    public static string[] Export(Environment environment) {
16        throw new NotImplementedException();
17    }
18 }
```

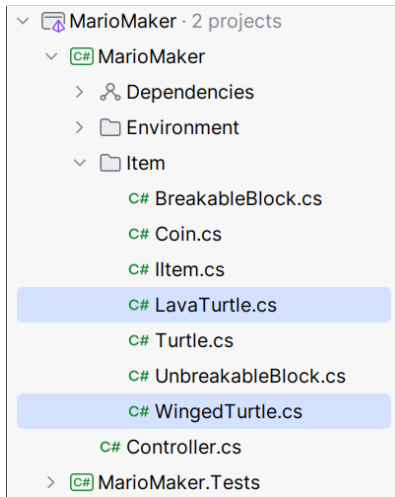
Solución Mario Maker



Solución Mario Maker

```
1 public class EnvironmentBuilder {
2     private Environment _environment;
3
4     public void Reset()
5         => _environment = new Environment();
6
7     public void SetDimensions(int height, int width)
8         => _environment.SetDimensions(height, width);
9
10    public void SetUnbreakableBlock(int row, int column)
11        => _environment.SetItem(row, column, new UnbreakableBlock());
12    public void SetBreakableBlock(int row, int column)
13        => _environment.SetItem(row, column, new BreakableBlock());
14    public void SetTurtle(int row, int column)
15        => _environment.SetItem(row, column, new Turtle());
16    public void SetCoin(int row, int column)
17        => _environment.SetItem(row, column, new Coin());
18
19    public Environment GetResult()
20        => _environment;
21 }
```


Solución Mario Maker



Solución Mario Maker

Esta es la tortuga de lava:

```
6 public class LavaTurtle : Turtle {  
7     public override string Export()  
8         => "LavaTurtle";  
9 }
```

Esta es la tortuga con alas:

```
11 public class WingedTurtle : Turtle{  
12     public override string Export()  
13         => "WingedTurtle";  
14 }
```

Solución Mario Maker

Esta es la tortuga de lava:

```
6 public class LavaTurtle : Turtle {  
7     public override string Export()  
8         => "LavaTurtle";  
9 }
```

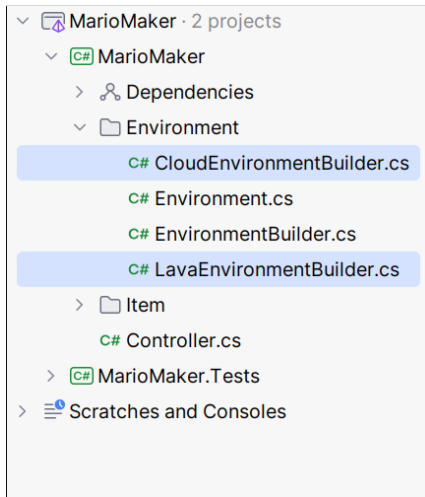
Esta es la tortuga con alas:

```
11 public class WingedTurtle : Turtle{  
12     public override string Export()  
13         => "WingedTurtle";  
14 }
```

... para que esto funcione tenemos que hacer que Export() virtual en Turtle:

```
1 public class Turtle : IItem {  
2     public virtual string Export() // Ahora es virtual!  
3         => "Turtle";  
4 }
```

Solución Mario Maker



Solución Mario Maker

Builder de lava (todo igual menos la tortuga):

```
1 public class LavaEnvironmentBuilder : EnvironmentBuilder {  
2     public override void SetTurtle(int row, int column)  
3         => Environment.SetItem(row, column, new LavaTurtle());  
4 }
```

Builder de nube (todo igual menos la tortuga):

```
6 public class CloudEnvironmentBuilder: EnvironmentBuilder {  
7     public override void SetTurtle(int row, int column)  
8         => Environment.SetItem(row, column, new WingedTurtle());  
9 }
```

Solución Mario Maker

Builder de lava (todo igual menos la tortuga):

```
1 public class LavaEnvironmentBuilder : EnvironmentBuilder {  
2     public override void SetTurtle(int row, int column)  
3         => Environment.SetItem(row, column, new LavaTurtle());  
4 }
```

Builder de nube (todo igual menos la tortuga):

```
6 public class CloudEnvironmentBuilder: EnvironmentBuilder {  
7     public override void SetTurtle(int row, int column)  
8         => Environment.SetItem(row, column, new WingedTurtle());  
9 }
```

... para que esto funcione tenemos que hacer cambios menores al builder base:

```
11 public class EnvironmentBuilder{  
12     protected Environment Environment; // Pasa a protected  
13     public virtual void SetTurtle(int row, int column) // Pasa a virtual  
14         => Environment.SetItem(row, column, new Turtle());  
15     /* ... el resto queda igual... */  
16 }
```

Ahora simplemente basta con retornar el builder apropiado en el Controller:

```
1 public static class Controller {  
2  
3     public static Environment BuildEnvironment(EnvironmentBuilder builder, string[]  
4         lines) {  
5         /* ... idem anterior ... */  
6     }  
7  
8     public static EnvironmentBuilder BuildCloudEnvironmentBuilder()  
9         => new CloudEnvironmentBuilder();  
10  
11    public static EnvironmentBuilder BuildLavaEnvironmentBuilder()  
12        => new LavaEnvironmentBuilder();  
13  
14    public static string[] Export(Environment environment) {  
15        throw new NotImplementedException();  
16    }  
17 }
```

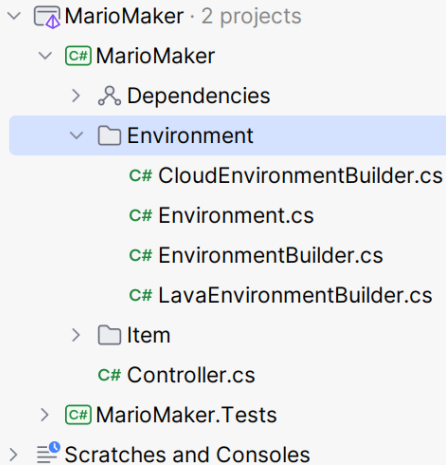
Parte c) Actualmente existe un programa para obtener la representación visual de los ambientes. Sin embargo, para ello necesitas exportar los niveles en un archivo de texto en donde cada línea representa una fila de la grilla. A su vez, cada fila se exporta mediante exportar el código de cada `Item` separado por un guión.

Los items que no representen una entidad del nivel se considerarán parte del fondo del ambiente y cada tipo de ambiente tiene un fondo distinto:

- Nivel original: "Sky".
- Nivel del mundo lava: "Lava".
- Nivel del mundo de nubes: "Cloud".

Utiliza el patrón `Visitor` para implementar el método `Export()` en `Controller.cs` para obtener su respectiva representación visual.

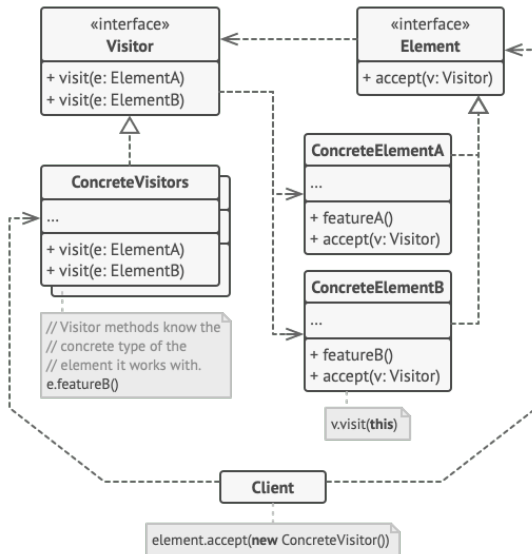
Solución Mario Maker



Visual Studio Solution Explorer showing the project structure for MarioMaker:

- ▼ MarioMaker · 2 projects
 - ▼ C# MarioMaker
 - > Dependencies
 - ▼ Environment
 - C# CloudEnvironmentBuilder.cs
 - C# Environment.cs
 - C# EnvironmentBuilder.cs
 - C# LavaEnvironmentBuilder.cs
 - > Item
 - C# Controller.cs
 - > C# MarioMaker.Tests
 - > Scratches and Consoles

Solución Mario Maker



Dependiendo del tipo de **ambiente** las casillas sin items se exportan distinto:

- Nivel original: “Sky”.
- Nivel del mundo lava: “Lava”.
- Nivel del mundo de nubes: “Cloud”.

... pero actualmente solo tenemos un tipo de ambiente (llamado Environment).

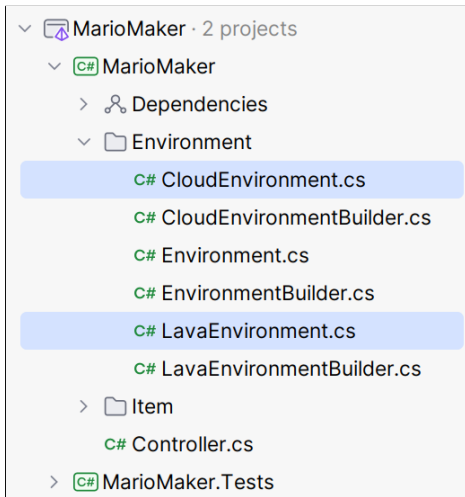
Dependiendo del tipo de **ambiente** las casillas sin items se exportan distinto:

- Nivel original: "Sky".
- Nivel del mundo lava: "Lava".
- Nivel del mundo de nubes: "Cloud".

... pero actualmente solo tenemos un tipo de ambiente (llamado Environment).

Partamos por crear ambientes de tipo lava y nube.

Solución Mario Maker



Por ahora, estos ambientes no hacen nada.

```
1 public class CloudEnvironment : Environment
2 {
3
4 }
```

Por ahora, estos ambientes no hacen nada.

```
1 public class CloudEnvironment : Environment
2 {
3
4 }
```

Este es el ambiente de lava:

```
6 public class LavaEnvironment : Environment
7 {
8
9 }
```

Por ahora, estos ambientes no hacen nada.

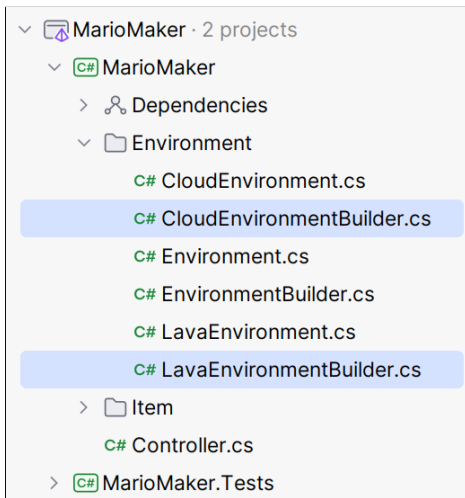
```
1 public class CloudEnvironment : Environment
2 {
3
4 }
```

Este es el ambiente de lava:

```
6 public class LavaEnvironment : Environment
7 {
8
9 }
```

Luego en cada builder necesitamos que se retorne el tipo de ambiente correcto.

Solución Mario Maker



Para que el builder retorne el ambiente correcto hay que sobrecribir `Reset()`.

```
1 public class EnvironmentBuilder {  
2     protected Environment Environment;  
3  
4     public virtual void Reset()  
5         => Environment = new Environment();  
6     /* ... */  
7 }
```

Solución Mario Maker

Para que el builder retorne el ambiente correcto hay que sobrescribir `Reset()`.

```
1 public class EnvironmentBuilder {  
2     protected Environment Environment;  
3  
4     public virtual void Reset()  
5         => Environment = new Environment();  
6     /* ... */  
7 }
```

Así queda el builder de nube:

```
1 public class CloudEnvironmentBuilder : EnvironmentBuilder {  
2     public override void SetTurtle(int row, int column)  
3         => Environment.SetItem(row, column, new WingedTurtle());  
4  
5     public override void Reset()  
6         => Environment = new CloudEnvironment();  
7 }
```

Solución Mario Maker

Para que el builder retorne el ambiente correcto hay que sobrescribir `Reset()`.

```
1 public class EnvironmentBuilder {  
2     protected Environment Environment;  
3  
4     public virtual void Reset()  
5         => Environment = new Environment();  
6     /* ... */  
7 }
```

Así queda el builder de lava:

```
1 public class LavaEnvironmentBuilder : EnvironmentBuilder {  
2     public override void SetTurtle(int row, int column)  
3         => Environment.SetItem(row, column, new LavaTurtle());  
4  
5     public override void Reset()  
6         => Environment = new LavaEnvironment();  
7 }
```

Solución Mario Maker

Para que el builder retorne el ambiente correcto hay que sobrescribir `Reset()`.

```
1 public class EnvironmentBuilder {  
2     protected Environment Environment;  
3  
4     public virtual void Reset()  
5         => Environment = new Environment();  
6     /* ... */  
7 }
```

Así queda el builder de lava:

```
1 public class LavaEnvironmentBuilder : EnvironmentBuilder {  
2     public override void SetTurtle(int row, int column)  
3         => Environment.SetItem(row, column, new LavaTurtle());  
4  
5     public override void Reset()  
6         => Environment = new LavaEnvironment();  
7 }
```

... ahora creamos un `Visitor` que permita exportar los tres tipos de ambientes.

```
1 public class Visitor {
2     public string[] ExportNormalEnv(Environment environment)
3         => Export(environment, "Sky");
4
5     public string[] ExportCloudEnv(CloudEnvironment environment)
6         => Export(environment, "Cloud");
7
8     public string[] ExportLavaEnv(LavaEnvironment environment)
9         => Export(environment, "Lava");
10
11     private string[] Export(Environment environment, string emptyKeyword) {
12         int height = environment.Height;
13         int width = environment.Width;
14         string[] result = new string[height];
15         for (int row = 0; row < height; row++) {
16             List<string> line = new List<string>();
17             for (int column = 0; column < width; column++) {
18                 IItem? item = environment.GetItem(row, column);
19                 line.Add(item == null ? emptyKeyword : item.Export());
20             }
21             result[row] = string.Join('-', line);
22         }
23         return result;
24     }
25 }
```

Luego tenemos que hacer que cada ambiente llame al método correcto del visitor.

```
1 public class Environment {  
2     public virtual string[] Export(Visitor visitor)  
3         => visitor.ExportNormalEnv(this);  
4     /* ... */  
5 }
```

Solución Mario Maker

Luego tenemos que hacer que cada ambiente llame al método correcto del visitor.

```
1 public class Environment {  
2     public virtual string[] Export(Visitor visitor)  
3         => visitor.ExportNormalEnv(this);  
4     /* ... */  
5 }
```

Ambiente nube llama al exportar de nube.

```
1 public class CloudEnvironment : Environment {  
2     public override string[] Export(Visitor visitor)  
3         => visitor.ExportCloudEnv(this);  
4 }
```


Solución Mario Maker

Luego tenemos que hacer que cada ambiente llame al método correcto del visitor.

```
1 public class Environment {  
2     public virtual string[] Export(Visitor visitor)  
3         => visitor.ExportNormalEnv(this);  
4     /* ... */  
5 }
```

Ambiente nube llama al exportar de nube.

```
1 public class CloudEnvironment : Environment {  
2     public override string[] Export(Visitor visitor)  
3         => visitor.ExportCloudEnv(this);  
4 }
```

Ambiente lava llama al exportar de lava.

```
1 public class LavaEnvironment : Environment {  
2     public override string[] Export(Visitor visitor)  
3         => visitor.ExportLavaEnv(this);  
4 }
```

Finalmente implementamos el método del Controller.cs:

```
1 public static class Controller {
2
3     public static Environment BuildEnvironment(EnvironmentBuilder builder, string[]
4         lines) {
5         /* ... */
6     }
7
8     public static EnvironmentBuilder BuildCloudEnvironmentBuilder()
9         => new CloudEnvironmentBuilder();
10
11    public static EnvironmentBuilder BuildLavaEnvironmentBuilder()
12        => new LavaEnvironmentBuilder();
13
14    public static string[] Export(Environment environment)
15        => environment.Export(new Visitor());
16 }
```